

Tutor Mx System

Gitea Sprint Handbook and Review Guide

Beginner-friendly Sprint 1-4 issue, testing, handover and deployment guide
for six Members

CURRENT STACK

React + Vite frontend | Auth0 | handwritten Node.js + Express API | Prisma | Neon PostgreSQL | Gitea main | GitHub deployment mirror | Codecov | Cloudflare Pages | Render API

COMS3011A Project 1 | 2026

Fully updated 30 September 2026

AI Declaration: This edition was updated with ChatGPT [GPT-5.6 SoI] from the supplied handbook, the supplied current Tutor Mx source archive and the team's recorded workflow. The team remains responsible for checking requirements, commands and evidence before submission.

UPDATED THROUGH 30 SEPTEMBER 2026

Sprint 1-2 history retained; Sprint 3 marked complete; shared timetable, Tutor sick-session handling and Master Organiser approval documented; Sprint 4 roadmap rebuilt around the latest source so work is not repeated.

What changed in this edition

- **Current source audit moved to 30 September 2026.** Pages 30 onward are rewritten from the supplied latest code archive. The archive contains no .git metadata, so this handbook does not invent a current SHA.
- **Sprint 3 is now a completed baseline, not future work.** Notifications/search, staffing/ranking/bulk allocation, reports, timetable import, timesheet correction/dispute/export, overflow management, API hardening and the Organiser Command Centre are treated as already built.
- **The timetable baseline is newer than the old handbook.** Tutor manual/import entry now shares one calendar, supports Class/Unavailable categories, Once/Weekly/Fortnightly recurrence, academic terms and occurrence-exception data.
- **Tutor sickness handling is already implemented.** A Tutor excuse keeps session history, supports organiser approve/reject, and records Excused/Skipped attendance state. Sprint 4 must not rebuild it.
- **Master Organiser approval is now the current Organiser registration method.** A lecturer registers first, remains pending with no Tutor Mx Organiser Profile, and receives ORGANISER only after Master Organiser approval.
- **New Sprint 4 student scope is added.** Students gain the same underlying timetable system, mutual Student/Tutor availability, tutoring bookings that appear on both calendars, and Student sick-note requests reviewed by Organisers.
- **Integration remains chronological.** Member 1 -> Integration Lead merges -> Member 2 pulls main -> repeat through Member 6. Merging is a workflow step, not a Member 1-6 feature card.

HISTORICAL PAGES 13-29

Sprint 1, Sprint 2 and post-Sprint-2 pages are retained as historical evidence. Where older Organiser sign-up wording differs, the current pages 4-6 supersede it. Do not reopen completed historical work unless a reproducible regression exists.

SPRINT 4 SCOPE CONTROL

Before coding any card, check current main first. If the feature already exists, narrow the card to the real remaining gap. No second timetable model, no second notification system, no second allocation rule set and no manual Auth0 subject entry.

How the team works now

Use this method in Sprint 1, 2, 3 and 4

NO DEVELOP BRANCH

Gitea main is the official shared branch. Members do normal feature work on their own branch. The Integration Lead reviews/integrates approved work into main.

Step	What to do
1. Refresh	git switch main -> git pull origin main
2. Create/use your branch	git switch -c sX/mY/short-name (or switch to your existing issue branch)
3. Work locally	Make only the changes for your issue. Keep secrets out of files/screenshots.
4. Test	Follow page 7. Then run the browser/local verification on page 8.
5. Commit your branch	Commit focused work on your branch. Do not commit directly to main.
6. Push + hand over	Push the branch to Gitea and tell the group: issue ID, branch, tests run, known limits, AI attribution.
7. Review + integrate	Reviewer checks the issue. Integration Lead integrates approved work into main and reruns the affected regression/full gate.
8. Refresh again	After main changes, everyone pulls the latest main before starting/resuming later work.

INTEGRATION LEAD EXCEPTION

When the Integration Lead performs integration/repository-maintenance work themselves, reviewed/tested changes may be committed through their integration workflow to main. This does not turn main into a normal Member working branch.

NEW SHARED-RUNNER STEP AFTER PUSH

After a Member pushes the branch, confirm the Gitea Actions workflow starts. If it is queued, wait for a shared runner to become free. Keep the completed run link/status with the handover evidence. The reviewer and Integration Lead check the CI result as part of review; local page 7-8 testing is still required.

SPRINT 3-4 CURRENT METHOD - ONE BRANCH PER MEMBER

For Sprint 3 and Sprint 4, each Member uses one branch for all of that Member's sprint work. Member 1 branches from current main, finishes all assigned work, pushes and hands over. The Integration Lead reviews/tests and merges that branch to main. Only then does Member 2 pull the updated main and create their one branch. Repeat through Member 6. Do not branch from the previous Member's feature branch and do not create separate branches for individual cards.

Architecture and current role onboarding

Layer	Beginner meaning
1. Browser	Loads Tutor Mx. Student and Tutor can create their normal account. Lecturers use Register as organiser.
2. Cloudflare Pages	Serves the React + Vite frontend. Only public VITE_* configuration belongs here.
3. Auth0	Owns identity, passwords, email verification and role claims. MASTER_ORGANIZER is an extra privilege, never a public sign-up choice.
4. React onboarding	Student/Tutor first-time accounts call POST /api/auth/onboarding. Lecturer Organiser registration calls POST /api/auth/organiser-registration instead.
5. Express on Render	Verifies Auth0, applies role/ownership rules and uses server-only Auth0 Management credentials when a Master Organiser approves a lecturer.
6. Prisma + Neon	Stores Profile/application/workflow data. Pending Organiser applicants have an OrganiserApplication but no Organiser Profile yet. Browser code never connects directly to Neon.
7. Later sign-ins	Student/Tutor/approved Organiser calls /api/me and opens the saved workspace. Pending/rejected Organiser applicants cannot enter another role workspace through that registration.

CURRENT ROLE RULE

Student and Tutor are normal Tutor Mx roles. Organiser is a lecturer role that now requires approval. MASTER_ORGANIZER is not a fourth workspace role; it is a trusted Auth0 privilege added to an existing ORGANISER account.

SECRETS RULE

DATABASE_URL and Auth0 Management API client secrets stay on the backend host. They must never appear in VITE_* variables, screenshots, issues, chat or committed files.

Very beginner guide: create an account or sign in

Action	Expected path
Create Student	Choose Student -> create account in Auth0 -> server onboarding -> Student Profile -> /student.
Create Tutor	Choose Tutor -> create account in Auth0 -> server onboarding -> Tutor Profile -> /tutor.
Register as Organiser	Choose Organiser -> Register as organiser -> Auth0 verifies lecturer identity -> pending OrganiserApplication. No Organiser Profile/role yet.
Check Organiser status	On the landing page, enter the lecturer email under Organiser -> Check status -> Pending / Approved / Rejected / Not found.
Approved Organiser signs in	After Master Organiser approval, sign in -> Auth0 token contains ORGANISER -> /api/me -> /organiser.
Existing Student/Tutor	Click Sign in -> Auth0 -> /api/me -> saved workspace. No second role choice.
Switch test role	Sign out, then use a separate test account created for that role. Do not change a real account role through normal onboarding.

EMAIL VERIFICATION

Auth0 remains responsible for email verification. An unverified account must not be silently linked to a Tutor Mx role or approved as an Organiser.

NORMAL SETUP IS AUTOMATIC

Do not manually create ordinary Student/Tutor Neon Profile rows. For Organisers, the one intentional admin step is Master Organiser approval. Do not give applicants ORGANISER manually as the normal workflow.

Master Organiser - beginner explanation for the team

Simple idea: the Master Organiser is an Organiser account with one extra trusted permission: deciding who is allowed to become an Organiser.

NOT A FOURTH ROLE
Users never choose Master Organiser on the landing page. The designated trusted lecturer is given both ORGANISER and MASTER_ORGANIZER once in Auth0 by an authorised project/admin owner.

Step	What happens in simple terms
1. Lecturer registers	Lecturer clicks Register as organiser and signs up/signs in through Auth0.
2. Waiting room	Tutor Mx stores a PENDING OrganiserApplication with the lecturer Auth0 identity, email and name. It does not create a Student Profile and does not grant ORGANISER yet.
3. Master sees queue	Only a signed-in ORGANISER whose Auth0 token also contains MASTER_ORGANIZER sees Organiser applications in the Organiser sidebar.
4. Approve	Backend rechecks Master permission, grants ORGANISER through the server-only Auth0 Management API, creates/updates the Neon Profile as ORGANISER and records reviewer/time.
5. Reject	Application becomes REJECTED. No Student/Organiser Profile is created for that application. Optional review note is stored.
6. Applicant checks status	Lecturer can check status from the landing page. Once Approved, they sign in normally and enter the Organiser workspace.

SECURITY THAT MEMBERS MUST PRESERVE
Hiding the menu is not enough. The backend routes also require ORGANISER + MASTER_ORGANIZER; a normal Organiser receives 403. Auth0 Management secrets stay on the backend. Never expose MASTER_ORGANIZER as a sign-up option.

ONE-TIME BOOTSTRAP
Auth0 Dashboard -> trusted main lecturer -> assign ORGANISER and MASTER_ORGANIZER -> sign out/in again so the new token contains both roles. After that, ordinary Organiser applications should be approved from Tutor Mx, not by manually editing each applicant in Auth0.

How to run the tests in VS Code / PowerShell

Every Member uses this page before handing work over

FAST RULE

From the repository root, npm run check is the current full local release-style gate. It runs Prisma generation, lint, backend type-checking, backend + frontend tests, production builds and the repository security check.

```
npm ci
npm ci --prefix frontend
npm run generate
npm run check
npm run coverage
```

Command	What it proves
npm run check	Full gate: generate -> lint -> typecheck -> tests -> build -> security check.
npm test	Backend + frontend automated tests.
npm run test:backend	Backend Vitest suite only.
npm run test:frontend	Frontend Vitest suite only.
npm run coverage	Backend + frontend coverage reports.
npm run lint	Backend + frontend lint.
npm run typecheck	Backend TypeScript check.
npm run build	Production backend + frontend builds.
npm run security:check	Tracked-file secret / repository safety check.

WHAT COUNTS AS PASSING

The command must exit successfully. Warnings are not the same as failed tests, but new warnings caused by your change should still be investigated and mentioned in the handover. Never hide a failed command.

AFTER THE LOCAL GATE PASSES

Commit and push the Member branch to Gitea, then confirm its Actions workflow runs on the shared lecturer runners. A queued job is not a failure; wait for it to auto-process. Save the run link/logs and Codecov result with the handover evidence.

How to check your work locally before the Integration Lead touches main

PURPOSE

This is the beginner smoke test Members run on their own branch. It answers: does my change actually work in the app before I tell the Integration Lead it is ready?

```
git switch main
git pull origin main
git switch <your-issue-branch>

# first clean setup / when dependencies changed
npm ci
npm ci --prefix frontend
npm run generate

# start frontend + backend together
npm run dev
```

- [] Open <http://localhost:5173>. The API normally runs at <http://localhost:3000>.
- [] Use the role/account relevant to your issue. For role testing, use separate Student, Tutor and Organiser test accounts.
- [] Repeat the acceptance path yourself: success case, invalid/failure case, and one relevant empty/permission case.
- [] Refresh the page and retry the action. Check that loading, error and success states remain usable.
- [] For protected flows, confirm another role cannot perform the action just by typing a URL or changing a request.
- [] Stop the dev servers with Ctrl+C. Then run `npm run check` and `npm run coverage`.
- [] Run `git status --short` and confirm only the files you intended to change are listed.

HANDOVER EVIDENCE

Send the issue ID + branch name + local commands/results + screenshot/API example where useful + known limitation. After pushing, also send the Gitea Actions run link/status and Codecov result once the shared runner finishes. Do not send secrets or production personal data.

AFTER PUSHING TO GITEA

Queued = normal on the shared runners; wait. Failed = investigate and rerun. Genuine runner/infrastructure outage = record the blocker plus local evidence and retry/contact staff. Never report CI as green unless the workflow actually completed successfully.

Lecturer update: shared Gitea Actions runners and CI evidence

WHAT CHANGED ON 1 SEPTEMBER 2026

The lecturer has set up two tested shared runners, sdp-runner-1 and sdp-runner-2, for all groups. Because they are shared, a job may sit in the queue and then auto-process when a runner becomes free. Local testing remains required.

WHAT WE MUST DO

- Before handover, run the page 7 automated checks and page 8 local acceptance/smoke checks.
- Push the Member branch to Gitea and let the Actions workflow run on the shared runners.
- If the job is queued, wait; queueing is expected when another group is using the runners.
- When the job completes, keep the run link/logs and Codecov result as CI evidence.
- If the workflow fails, investigate the code/configuration and rerun it. If the runner infrastructure genuinely remains unavailable, attach local evidence, record the blocker honestly and contact staff for persistent/complex runner issues.
- Do not weaken/remove tests and do not claim CI or Codecov is green when the workflow did not complete.
- A private runner is optional. If the group registers one, use a descriptive name such as <group-name>-runner-`<id>`; the lecturer may remove private runners for admin/security reasons.

RUNNER LIMITS AND QUEUE RULE

Shared runners are limited to 4 GB memory and currently time out after 3 hours; the lecturer may reduce the timeout to 30 minutes later. Keep workflows reasonably simple. Queued is not failed - wait for auto-processing.

Member handover -> Integration Lead -> Gitea main -> GitHub mirror

```
git status
git add <only-your-files>
git diff --cached --check
git commit -m "<clear issue-focused message>"
git push -u origin <your-issue-branch>
```

- Member: do not merge your own normal feature branch into main. Run local checks first, commit, and push the issue branch.
- After push: confirm Gitea Actions starts. If queued, wait for a shared runner; once complete, attach the run link/status and Codecov result to the handover.
- Reviewer: reproduce the issue acceptance checks, inspect changed files, and check both local evidence and the completed CI result.
- Integration Lead: pull latest main, integrate approved work, resolve conflicts with the author, rerun npm run check (plus coverage/database checks when relevant), then push Gitea main and verify the main-branch Actions result if the workflow triggers there.
- Deployment mirror: only after Gitea main is approved, sync that source to the GitHub deployment repository. GitHub is not the team source of truth.

BEFORE EVERY LATER TASK

After an integration changes main, pull main again before continuing Sprint 2/3/4 work. This prevents members from building on an old snapshot.

SPRINT 3-4 HANDOVER RULE

After each Member finishes their single Sprint 3/4 branch, the Integration Lead reviews/tests and merges it to Gitea main. The next Member must pull that updated main before creating their own single branch. This repeats Member 1 -> 2 -> 3 -> 4 -> 5 -> 6.

Production deployment after approved integration

RELEASE SOURCE

Gitea main is the team source of truth. Only approved main is mirrored/released. Member branches do not deploy production.

Frontend - Cloudflare Pages

From the approved repository root in PowerShell:

COMMANDS

```
git switch main
git pull origin main
cd frontend
Remove-Item -Recurse -Force dist -ErrorAction SilentlyContinue
npm run build
npx wrangler pages deploy dist --project-name=tutor-mx --branch=main --commit-dirty=true
```

Backend - Render

- Deploy/sync the approved backend source using the team Render workflow.
- Apply new Prisma migrations to Neon before running backend code that expects the new schema.
- Keep DATABASE_URL and Auth0 Management credentials only in Render/server environment settings.

Live verification

SMOKE CHECK

```
Invoke-WebRequest "https://tutor-mx-api.onrender.com/health" -UseBasicParsing
Invoke-WebRequest "https://tutor-mx-api.onrender.com/ready" -UseBasicParsing
```

Both should return HTTP 200 when the API and database are healthy. Then open tutor-mx.pages.dev, hard refresh, and run the affected Student/Tutor/Organiser flow.

VITE RULE

VITE_* values are embedded at frontend build time. If production frontend environment values change, rebuild dist before deploying it.

AI policy and Global Definition of Done

AI policy

- [] Remove secrets, tokens and personal data before using an AI tool.
- [] Read, understand and test AI-generated code before committing it.
- [] If a commit contains AI-generated code, use the course-required Assisted-by: <tool>[<model>] attribution with the actual tool/model.
- [] Keep README/document AI declarations current. Never invent test evidence, stakeholder evidence or tool/model names.

Global Definition of Done - applies to every issue in Sprint 1-4

- [] The issue-specific acceptance checks pass.
- [] Page 8 local browser/behavior check is completed for the affected flow.
- [] Page 7 automated test gate is run; relevant coverage is produced.
- [] Follow page 9: wait for queued shared-runner CI, keep completed run evidence, and document genuine infrastructure blockers honestly.
- [] Only intended files are committed on the Member branch; no secrets/personal data are added.
- [] The branch is pushed to Gitea and the assigned reviewer/Integration Lead is told it is ready.
- [] The Member does not merge normal feature work to main. Integration Lead integrates approved work and reruns regression.
- [] AI attribution/declarations are correct.

Sprint 1: plan, order and handovers

3-25 August 2026 | Integrated foundation, public demo and one secure vertical path

SPRINT 1 IS THE BASELINE

Current main already contains the integrated Member 1-6 Sprint 1 foundation. Use these issue cards for traceability, regression and follow-up fixes; do not rebuild working foundations.

- Keep the automatic Auth0 -> Express onboarding -> Neon Profile method as the normal account path.
- Keep Organiser Tutor Management registered-tutor based; no Auth0 subject field.
- Keep Tutor routes own-identity based; no browser tutor-ID ownership.
- Keep Student overflow/volunteer on the Sprint 1 mock boundary until Sprint 2 real endpoints are integrated.
- Keep Save allocation as the Sprint 1 placeholder until Sprint 2 allocation-write work is ready.

Member	Sprint ownership
Member 1	Authentication, role security, notifications/search and shared release checks
Member 2	Organiser interface, allocation board, approvals, reports and scenarios
Member 3	Tutor dashboard, availability, timesheets, excuses, imports and swaps
Member 4	Student overflow plus shared responsive and accessible interface
Member 5	Handwritten Express API, validation, business rules, security and API tests
Member 6	Neon PostgreSQL, Prisma migrations, external integration, data engine, CI/deployment

EVERY MEMBER

Before handover: use page 7 for automated tests, page 8 for local browser verification, page 9 for the lecturer-runner rule, and page 12 for the Global Definition of Done.

Sprint 1 - Member 1

Authentication, role security, notifications/search and shared release checks

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

SPRINT 1 BASELINE NOTE

This Member area is already part of the integrated Sprint 1 baseline. Use it to verify/repair the current behavior; do not recreate a second implementation of the same foundation.

S1-M1-1 | Self-service Auth0 sign-up, role onboarding and direct routing

Branch: s1/m1/auth-entry | Reviewer: Member 2 | 4 points | Integrates to: Gitea main

Goal: As a new or returning user, I want Tutor Mx to link my authenticated identity to exactly one role and open the correct workspace without manual setup.

Acceptance checks

- A new user chooses Student, Tutor or Organiser before Auth0 sign-up; the server creates the matching Tutor Mx role/Profile automatically.
- An existing linked user signs in once and is routed directly to /student, /tutor or /organiser without choosing a role again.
- An unverified email or onboarding conflict is handled safely; passwords and Management API credentials never enter Tutor Mx browser code.

Member tasks

- [] Maintain the role-choice entry screen and Auth0 Universal Login flow.
- [] Use POST /api/auth/onboarding for first-time linking and /api/me for later sign-ins.
- [] Keep sign-out available from all three workspaces and cover role-routing failures with tests.

Handoff: Give Member 5 the bearer-token shape and give Members 2-4 the signed-in user/role interface.

S1-M1-2 | Account settings, password reset, deletion and route protection

Branch: s1/m1/auth-lifecycle | Reviewer: Member 2 | 4 points | Integrates to: Gitea main

Depends on: S1-M1-1 and role names Organiser/Tutor/Student

Goal: As a Tutor Mx user, I want safe account recovery and protected role navigation so that I can manage access without exposing private data.

Acceptance checks

- Password-reset requests use a safe non-enumerating response and Auth0 owns the actual password workflow.
- Account deletion requires confirmation and the privileged Auth0/database deletion runs only on the backend.
- Wrong-role or unlinked access is blocked even if a user manually types a protected URL.

Member tasks

- [] Maintain Account Settings reset/deletion actions and confirmation states.
- [] Keep route guards aligned with backend role/ownership checks.
- [] Test cancel/confirm, wrong-role, missing-profile and signed-out paths.

Handoff: Give Member 5 the role matrix and give the Project Coordinator screenshots/test evidence for the security guide.

S1-M1-3 | Authentication contract and permission tests

Branch: s1/m1/auth-contract-tests | Reviewer: Member 2 | 4 points | Integrates to: Gitea main

Depends on: S1-M1-1, S1-M1-2 and S1-M5-2

Goal: As the team, we want one tested bearer-token and role contract so that every feature uses the same secure identity rules.

Acceptance checks

- Protected routes return 401 for missing/invalid tokens and 403 for valid users with the wrong role.
- A permitted token reaches the handler only when its Tutor Mx role matches the linked Neon Profile.
- Fresh first-login onboarding is handled safely even when the first token predates the newly assigned role.

Member tasks

- [] Maintain the shared bearer-token request helper.
- [] Keep backend Auth0 issuer/audience/JWKS/expiry checks covered.
- [] Add contract tests whenever a new protected route is introduced.

Handoff: Member 5 consumes the token helper/fixtures; Members 2-4 use the request wrapper rather than copying auth code.

Sprint 1 - Member 2

Organiser interface, allocation board, approvals, reports and scenarios

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

SPRINT 1 BASELINE NOTE

This Member area is already part of the integrated Sprint 1 baseline. Use it to verify/repair the current behavior; do not recreate a second implementation of the same foundation.

S1-M2-1 | Organiser workspace and allocation-board foundation

Branch: s1/m2/organiser-board-shell | Reviewer: Member 1 | 4 points | Integrates to: Gitea main

Depends on: Shared navigation from S1-M4-2; mock candidate contract from

Goal: As an organiser, I want one clear workspace for courses, tutors and allocation checks so that management work is easy to follow.

Acceptance checks

- The Organiser workspace loads only for the ORGANISER role and keeps loading/empty/error states usable.
- Candidate mark, clash and hour results are visible together before selection.
- The workspace remains keyboard reachable and usable on common phone widths.

Member tasks

- [] Maintain the Organiser navigation and allocation-board shell.
- [] Keep candidate data and rule-result components aligned with the API contract.
- [] Preserve clear loading, empty, error and retry states.

Handoff: Give Member 5 a list of required fields and give Member 4 any reusable table/card needs.

S1-M2-2 | Course management and registered-tutor management UI

Branch: s1/m2/course-tutor-ui | Reviewer: Member 1 | 4 points | Integrates to: Gitea main

Depends on: S1-M2-1 and S1-M5-2 contract

Goal: As an organiser, I want to manage courses and already-registered tutors using normal account information instead of technical Auth0 identifiers.

Acceptance checks

- Signed-up Tutor accounts appear from the database automatically and can be searched by name or email.
- The Organiser never has to enter, copy or see an Auth0 subject/User ID in the normal Tutor Management UI.
- Edit uses the internal tutor database record to maintain details, weekly-hour limit, marks and busy times.

Member tasks

- [] Maintain course add/edit/remove UI with validation and confirmations.
- [] Maintain tutor search/list/edit UI for self-onboarded Tutor profiles.
- [] Keep the interface professional: no implementation-level Auth0/Neon instructions in normal organiser screens.

Handoff: Member 6 uses the final field list in migrations; Member 5 matches the request/response shapes.

S1-M2-3 | Immediate allocation-rule messages and organiser UI tests

Branch: s1/m2/allocation-rule-ui | Reviewer: Member 1 | 4 points | Integrates to: Gitea main

Depends on: S1-M2-1 and rule outputs from S1-M5-3

Goal: As an organiser, I want mark, timetable and remaining-hour checks explained before saving so that invalid choices are obvious.

Acceptance checks

- Mark threshold, clash and remaining-hours checks each show written Pass/Fail or controlled unknown wording.
- Save remains unavailable until the selected tutor passes the required checks.
- Colour is never the only way a rule result is communicated.

Member tasks

- [] Maintain the reusable rule-result component.
- [] Test pass/fail/unknown and boundary values.
- [] Keep candidate refresh and selected-tutor state stable after errors.

Handoff: Member 5 supplies stable reason codes; Member 2 maps those codes to plain language without duplicating business logic.

Sprint 1 - Member 3

Tutor dashboard, availability, timesheets, excuses, imports and swaps

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

SPRINT 1 BASELINE NOTE

This Member area is already part of the integrated Sprint 1 baseline. Use it to verify/repair the current behavior; do not recreate a second implementation of the same foundation.

S1-M3-1 | Tutor dashboard using the signed-in tutor identity

Branch: s1/m3/tutor-dashboard | Reviewer: Member 4 | 4 points | Integrates to: Gitea main

Depends on: Role routing from S1-M1-2 and shared states from S1-M4-2

Goal: As a tutor, I want my own assignments and weekly-hour summary without entering a tutor ID so that another tutor cannot be selected by changing a URL.

Acceptance checks

- GET /api/tutor/dashboard derives the tutor from the verified token subject and returns only that tutor's data.
- A newly onboarded Tutor with no hour limit or allocations gets a usable no-work state rather than an error.
- Loading, empty, error and retry states remain clear.

Member tasks

- [] Maintain the protected tutor dashboard route and cards/table.
- [] Display unset weekly limits safely and never request another tutor by browser-supplied ID.
- [] Test own-data, no-work and failed-load cases.

Handoff: Give Member 5 the dashboard fields and Member 6 the query/seed-data needs.

S1-M3-2 | Tutor availability, work-log and excuse forms on real API routes

Branch: s1/m3/tutor-forms | Reviewer: Member 4 | 4 points | Integrates to: Gitea main

Depends on: S1-M3-1 and initial API contracts

Goal: As a tutor, I want to record busy times, work logs and excuses through my own authenticated routes so that my records stay accurate.

Acceptance checks

- Tutor workflow endpoints derive ownership from the token and ignore browser-supplied tutorId values.
- Invalid or impossible time/hour values are rejected with clear field messages.
- Valid actions persist through the handwritten Express API and Prisma.

Member tasks

- [] Maintain availability/time-slot form and validation.
- [] Maintain work-log and excuse submission forms.
- [] Test valid, invalid, duplicate/state and cross-user protection cases.

Handoff: Member 5 receives the payload shapes; Member 6 receives required tables and constraints.

S1-M3-3 | Tutor UI contract, responsiveness and accessibility tests

Branch: s1/m3/tutor-ui-tests | Reviewer: Member 4 | 4 points | Integrates to: Gitea main

Depends on: S1-M3-1, S1-M3-2 and S1-M4-2

Goal: As the team, we want regression tests around the tutor journey so that later changes do not break the working dashboard and forms.

Acceptance checks

- Every required tutor control remains keyboard reachable.
- Required actions are not clipped at the agreed narrow/mobile width.
- Contract or validation changes that break the current tutor journey fail tests.

Member tasks

- [] Keep dashboard/form component tests current.
- [] Cover labels, focus order and responsive layout.
- [] Run the shared test/coverage gate before handover.

Handoff: Give the Project Coordinator the test command/output and give Member 5 any contract failures as bugs.

Sprint 1 - Member 4

Student overflow plus shared responsive and accessible interface

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

SPRINT 1 BASELINE NOTE

This Member area is already part of the integrated Sprint 1 baseline. Use it to verify/repair the current behavior; do not recreate a second implementation of the same foundation.

S1-M4-1 | Student overflow protected UI shell (Sprint 1 mock boundary)

Branch: s1/m4/student-overflow | Reviewer: Member 3 | 4 points | Integrates to: Gitea main

Depends on: Role routing from S1-M1-2

Goal: As a student, I want a clear protected overflow list so that I can understand unclaimed work while the real Student API is completed later.

Acceptance checks

- Only STUDENT users can open the page.
- Open, closed, empty and failed-load mock states are clear.
- The browser uses the agreed mock/API boundary and never talks directly to Neon.

Member tasks

- ☐ Maintain the Student overflow route/list/cards.
- ☐ Preserve open/closed/empty/error states and phone layout.
- ☐ Keep the mock contract stable until Sprint 2 connects the real backend endpoint.

Handoff: Give Member 5 and Member 6 the exact work-item fields needed by the UI.

S1-M4-2 | Shared role navigation and status components

Branch: s1/m4/shared-navigation | Reviewer: Member 3 | 4 points | Integrates to: Gitea main

Depends on: Role map from S1-M1-2

Goal: As any Tutor Mx user, I want consistent role navigation and status messages so that the app feels like one product.

Acceptance checks

- Only navigation permitted for the signed-in role is shown.
- Loading, empty, error, success and confirmation states use consistent accessible components.
- Sign out is reachable from Organiser, Tutor and Student workspaces.

Member tasks

- ☐ Maintain shared navigation/status components.
- ☐ Keep visible focus and skip-to-content behavior.
- ☐ Test role visibility and responsive navigation.

Handoff: Members 2 and 3 consume these components; Member 1 supplies the authoritative role list.

S1-M4-3 | Student volunteer confirmation and shared UI tests (Sprint 1 mock boundary)

Branch: s1/m4/volunteer-confirmation | Reviewer: Member 3 | 4 points | Integrates to: Gitea main

Depends on: S1-M4-1 and S1-M4-2

Goal: As a student, I want a clear confirm/cancel volunteer interaction so that duplicate or accidental claims are prevented.

Acceptance checks

- Confirmation repeats the selected work item and Cancel sends no request.
- Pending state blocks duplicate submission and success/failure is explained.
- Sprint 1 behavior remains on the agreed mock boundary until Sprint 2 real endpoints are ready.

Member tasks

- ☐ Maintain confirm/cancel and pending states.
- ☐ Test duplicate, already-claimed and failure paths.
- ☐ Hand the real request/error expectations to Sprint 2 backend work.

Handoff: Member 5 receives expected duplicate/error responses; the Project Coordinator receives test evidence.

Sprint 1 - Member 5

Handwritten Express API, validation, business rules, security and API tests

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

SPRINT 1 BASELINE NOTE

This Member area is already part of the integrated Sprint 1 baseline. Use it to verify/repair the current behavior; do not recreate a second implementation of the same foundation.

S1-M5-1 | Separate handwritten Express backend and health endpoint

Branch: s1/m5/backend-foundation | Reviewer: Member 6 | 4 points | Integrates to: Gitea main

Depends on: Chosen backend runtime and repository structure

Goal: As the team, we want a separately runnable handwritten API so that browser code never depends on direct database access.

Acceptance checks

- Backend starts independently and GET /health returns a documented safe response.
- Unhandled routes/errors use the shared safe error shape.
- Environment validation/logging never exposes secrets.

Member tasks

- ☐ Maintain the Express server, central error handling and health/readiness checks.
- ☐ Keep backend lint/test/build scripts working from the repository root.
- ☐ Cover health and error behavior with tests.

Handoff: Give Member 6 the environment and database-connection requirements; give all frontend Members the base URL.

S1-M5-2 | Auth middleware and core profile/course/tutor/tutor-workflow routes

Branch: s1/m5/core-api-contracts | Reviewer: Member 6 | 4 points | Integrates to: Gitea main

Depends on: S1-M5-1, auth contract from Member 1 and schema from

Goal: As the application, we want protected handwritten endpoints so that all role data reaches React through one authorised API.

Acceptance checks

- Bearer verification and role/ownership middleware protect core routes.
- Course/tutor organiser routes and own-tutor dashboard/forms use Prisma on the backend only.
- Tutor workflow routes derive identity from the token, not a browser tutor ID.

Member tasks

- ☐ Maintain the current-profile, course, tutor and tutor-workflow routes.
- ☐ Keep success/error JSON and status codes consistent.
- ☐ Add success, validation, 401, 403, 404 and conflict tests when contracts change.

Handoff: Members 1-4 use the published route/payload examples; Member 6 aligns migrations and seed data.

S1-M5-3 | Three allocation-rule functions and boundary tests

Branch: s1/m5/allocation-rules | Reviewer: Member 6 | 4 points | Integrates to: Gitea main

Depends on: Threshold decisions from client/coordinator and Member 6 data

Goal: As an organiser, I want server-side mark, clash and weekly-hour rules so that invalid allocations cannot be bypassed by the UI.

Acceptance checks

- Mark threshold and exact weekly limit follow the agreed inclusive boundaries.
- Touching time ranges are not a clash; true overlaps are.
- Missing/malformed data returns controlled unknown/failure rather than silent approval.

Member tasks

- ☐ Maintain pure rule functions and stable reason codes.
- ☐ Keep boundary tests for threshold, overlap and exact/exceeded hours.
- ☐ Ensure UI consumes server results instead of duplicating business logic.

Handoff: Member 2 maps reason codes to UI text; Member 6 provides candidate/availability/hour data.

Sprint 1 - Member 6

Neon PostgreSQL, Prisma migrations, external integration, data engine, CI/deployment

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

SPRINT 1 BASELINE NOTE

This Member area is already part of the integrated Sprint 1 baseline. Use it to verify/repair the current behavior; do not recreate a second implementation of the same foundation.

S1-M6-1 | Neon PostgreSQL schema, Prisma migrations and safe seed data

Branch: s | Reviewer: assigned reviewer | 4 points | Integrates to: Gitea main

Depends on: Field lists from Members 1-5

Goal: As the team, we want repeatable migrations and safe demo data so that every environment has the same data rules.

Acceptance checks

- Migrations create the required role/profile/course/mark/availability/hour/allocation structures from a clean database.
- Constraints reject invalid relationships and values.
- Seed data is invented/non-personal and no DATABASE_URL is committed.

Member tasks

- [] Maintain versioned Prisma migrations and seed data.
- [] Run schema/database verification after changes.
- [] Coordinate new fields with API and UI owners before migration changes land.

Handoff: Member 5 consumes the backend database connection; Members 2-4 receive stable demo identifiers and fields.

S1-M6-2 | Gitea integration checks, public deployment and shared Sprint 1 URL

Branch: s1/m6/ci-codecov | Reviewer: Member 5 | 4 points | Integrates to: Gitea main

Depends on: Test scripts from Members 1-5 and repository admin access

Goal: As the team, we want one tested Gitea main plus a public demo so that everyone works from the same source and can demonstrate the integrated system.

Acceptance checks

- Gitea main is the official shared source; members work on branches and the Integration Lead integrates approved work.
- The public frontend loads at tutor-mx.pages.dev and calls the hosted Express API rather than Neon directly.
- Local tests remain required; pushed branches use the shared Gitea Actions runners, and queued jobs are allowed to auto-process.

Member tasks

- [] Maintain Gitea workflow files and honest CI/runner evidence.
- [] Sync approved Gitea main to the GitHub deployment mirror when releasing.
- [] Build the frontend and manually upload the fresh frontend/dist folder to Cloudflare Production when required.

Handoff: All Members use the same test script names; the Integration Lead uses these checks as the merge gate.

S1-M6-3 | South African public-holiday API spike and fallback contract

Branch: s1/m6/external-api-spike | Reviewer: Member 5 | 4 points | Integrates to: Gitea main

Depends on: Client/coordinator chooses a relevant service and confirms

Goal: As Tutor Mx, we want a safe external public-holiday source to support scheduling without making the core system depend on that service.

Acceptance checks

- The backend-only spike returns a safe subset of South African holiday data.
- Timeout/unavailable/invalid responses return a controlled fallback.
- No user identity, token or database secret is sent to the external service.

Member tasks

- [] Maintain the Nager.Date holiday adapter/spike contract.
- [] Keep mocked success/timeout/invalid-response tests deterministic.
- [] Carry production route/display integration into later sprints rather than redoing the spike.

Handoff: Member 5 will integrate the adapter into the handwritten API; Member 4 will later display the safe response/fallback.

Sprint 1: integration and review gate

- [] Every Member starts from the latest Gitea main and hands over only their own tested branch.
- [] Assigned reviewer reproduces the issue-specific checks and checks page 7/8 evidence.
- [] Integration Lead integrates approved branches into main one at a time and reruns the affected regression/full gate.
- [] After a branch/main push, confirm Gitea Actions completes on a shared runner. A queued job is normal; wait for it. If infrastructure genuinely remains unavailable, record the blocker plus local evidence and do not mark CI green.
- [] After integration, run the relevant three-role/public-environment smoke test.
- [] Update the GitHub deployment mirror and manual Cloudflare Production artifact only from approved Gitea main when a release is required.
- [] Issue is Done only when integrated behavior passes; code merely existing on a Member branch is not Done.

SPRINT 1 SPECIFIC RELEASE CHECK

Verify new Student/Tutor/Organiser account creation -> Auth0 -> automatic onboarding -> correct role dashboard. Verify Organiser Tutor Management contains no Auth0-subject input.

Sprint 2: plan, order and handovers

26 August-15 September 2026 | Complete Basic journeys without rebuilding Sprint 1 foundations

DO NOT REPEAT EARLIER SPRINTS

Sprint 2 starts from the latest approved Gitea main. Do not recreate Auth0 onboarding, core role routing, existing schema/API/UI foundations or manual tutor-ID workflows. Add only the new scope and hardening listed below.

- Finish the remaining Basic end-to-end gaps: Student real API, allocation writes, approvals/statuses and hardening.
- Reuse the working automatic account linking and registered-tutor management from Sprint 1.
- Run representative-user and three-role regression before release.

Member	Sprint ownership
Member 1	Authentication, role security, notifications/search and shared release checks
Member 2	Organiser interface, allocation board, approvals, reports and scenarios
Member 3	Tutor dashboard, availability, timesheets, excuses, imports and swaps
Member 4	Student overflow plus shared responsive and accessible interface
Member 5	Handwritten Express API, validation, business rules, security and API tests
Member 6	Neon PostgreSQL, Prisma migrations, external integration, data engine, CI/deployment
EVERY MEMBER Before handover: use page 7 for automated tests, page 8 for local browser verification, page 9 for the lecturer-runner rule, and page 12 for the Global Definition of Done.	

Sprint 2 - Member 1

Authentication, role security, notifications/search and shared release checks

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

CARRY FORWARD

Start from the latest Gitea main and reuse the working Sprint 1 implementation. The three cards below are the new work for this sprint.

S2-M1-1 | Auth lifecycle hardening and email-verification recovery

Branch: s2/m1/account-lifecycle | Reviewer: Member 3 | 4 points | Integrates to: Gitea main

Depends on: Sprint 1 auth foundation and deployed backend

Goal: As a Tutor Mx user, I want edge cases around verification, reset, sign-out and deletion handled reliably without repeating the working Sprint 1 onboarding flow.

Acceptance checks

- A linked existing account never returns to role choice after normal sign-in.
- An unverified email gets a clear retry/recovery path; add resend-verification support if the agreed Auth0 approach allows it.
- Reset/deletion/network failure cannot leave the app in a false signed-in or half-deleted state.

Member tasks

- [] Keep Sprint 1 self-service onboarding unchanged unless a defect is proven.
- [] Harden verification/retry, reset, deletion and post-action session cleanup.
- [] Add integration tests for failure/recovery paths.

Handoff: Members 2-4 receive a stable signed-in session/role API; Coordinator receives acceptance evidence.

S2-M1-2 | Role and ownership protection across the completed Basic API

Branch: s2/m1/role-protection | Reviewer: Member 3 | 4 points | Integrates to: Gitea main

Depends on: S2-M1-1 and Member 5 Basic endpoints

Goal: As the school, we want every real Basic endpoint checked for role and ownership so that hidden navigation can never be bypassed.

Acceptance checks

- Organisator-only writes return 403 for Tutor/Student callers.
- Own-data routes do not reveal another Tutor/Student when identifiers are manipulated.
- Frontend route guards and backend authorisation remain consistent.

Member tasks

- [] Create an endpoint-to-role/ownership matrix for the now-real Basic routes.
- [] Add 401/403/cross-user tests for new Sprint 2 endpoints.
- [] Fix permission defects without moving security logic into React.

Handoff: Member 5 receives failing endpoint cases; Members 2-4 use the shared permission/error behavior.

S2-M1-3 | Cross-role security smoke suite and assigned fixes

Branch: s2/m1/security-smoke | Reviewer: Member 3 | 4 points | Integrates to: Gitea main

Depends on: All Sprint 2 Basic journeys integrated on Gitea main

Goal: As the Integration Lead, we want a repeatable three-role smoke suite so that integration cannot break access or expose another role's data.

Acceptance checks

- Separate Student/Tutor/Organisator test accounts land only in their expected workspace.
- Critical endpoints show consistent 401/403 behavior for missing, expired and wrong-role tokens.
- Any severity-one/two security defect blocks release until fixed or formally recorded as a blocker.

Member tasks

- [] Create/run the cross-role smoke sequence against the shared environment.
- [] Open reproducible bugs for failures and fix Member 1-owned defects.
- [] Attach local test evidence plus the completed Gitea Actions run link/status; if queued, wait for the shared runner.

Handoff: Integration Lead runs this before the Integration Lead release decision; Coordinator links results into testing evidence.

Sprint 2 - Member 2

Organiser interface, allocation board, approvals, reports and scenarios

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

CARRY FORWARD

Start from the latest Gitea main and reuse the working Sprint 1 implementation. The three cards below are the new work for this sprint.

S2-M2-1 | Organiser course/tutor management hardening - registered tutors only

Branch: s2/m2/organiser-crud | Reviewer: Member 5 | 4 points | Integrates to: Gitea main

Depends on: Member 5 organiser endpoints and Member 6 Basic schema

Goal: As an organiser, I want the working Sprint 1 management screens hardened around real registered Tutor accounts instead of rebuilding tutor creation.

Acceptance checks

- Tutor search/list comes from self-onboarded Tutor profiles and never asks for Auth0 subject/User ID.
- Edit/validation/refresh behavior stays correct after API failures.
- Course CRUD and tutor details/hour-limit/marks/busy-time edits persist and refresh cleanly.

Member tasks

- ☐ Do not rebuild Sprint 1 Tutor Management or add a manual tutor-create form.
- ☐ Fix validation, empty/error states, responsive layout and stale-data issues found in shared testing.
- ☐ Add integration tests around real course/tutor edit flows.

Handoff: Member 5 fixes contract mismatches; Member 6 validates data constraints.

S2-M2-2 | Live allocation create, edit and remove with server checks

Branch: s2/m2/allocation-board-basic | Reviewer: Member 5 | 4 points | Integrates to: Gitea main

Depends on: S2-M2-1, Member 5 allocation endpoints and Member 6 queries

Goal: As an organiser, I want the Sprint 1 allocation placeholder replaced by real save/edit/remove behavior while keeping all rule checks visible.

Acceptance checks

- A valid candidate can be saved through a real allocation-write endpoint.
- Failed mark/clash/hour rules are rejected server-side and remain visible in the UI.
- Editing/removing an allocation updates hours and assignments without manual database edits.

Member tasks

- ☐ Connect real allocation create/edit/remove endpoints.
- ☐ Refresh candidate hours and assignment data after every successful change.
- ☐ Test each failed rule plus valid create/edit/remove.

Handoff: Member 5 enforces rules on server; Member 3 sees resulting tutor obligations.

S2-M2-3 | Organiser timesheet and overflow approvals with tests

Branch: s2/m2/organiser-approvals | Reviewer: Member 5 | 4 points | Integrates to: Gitea main

Depends on: Member 3 timesheets, Member 4 volunteer requests and

Goal: As an organiser, I want to approve or return tutor timesheets and approve or reject student volunteer claims so that Basic workflows can complete end to end.

Acceptance checks

- Pending timesheets and claims show the information needed for a decision.
- Approve/reject/return actions persist a clear status and update the responsible user view.
- Duplicate or stale decisions are rejected safely.

Member tasks

- ☐ Build approval queues/views on the Organiser workspace.
- ☐ Connect to the Sprint 2 workflow endpoints and confirmation states.
- ☐ Test valid, rejected, stale and API-failure cases.

Handoff: Members 3 and 4 consume the resulting statuses; Coordinator receives demo-ready accounts/data.

Sprint 2 - Member 3

Tutor dashboard, availability, timesheets, excuses, imports and swaps

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

CARRY FORWARD

Start from the latest Gitea main and reuse the working Sprint 1 implementation. The three cards below are the new work for this sprint.

S2-M3-1 | Tutor dashboard hardening and live assignment refresh

Branch: s2/m3/tutor-dashboard-api | Reviewer: Member 1 | 4 points | Integrates to: Gitea main

Depends on: Member 5 own-dashboard endpoint and Member 2 allocations

Goal: As a tutor, I want the already-live Sprint 1 dashboard to stay accurate when allocations change and when my weekly limit is unset or updated.

Acceptance checks

- The dashboard shows only the signed-in tutor and refreshes after allocation changes.
- No-work and no-hour-limit states remain usable.
- Failed refresh preserves the last safe state and offers retry.

Member tasks

- [] Do not replace the working own-tutor API with a new tutor-ID flow.
- [] Add refresh/status handling for allocation changes.
- [] Expand dashboard tests for changed assignments, unset limits and retry.

Handoff: Member 2 confirms allocation changes; Member 5 fixes data/permission contract issues.

S2-M3-2 | Complete timesheet submission workflow from existing work logs

Branch: s2/m3/timesheet-basic | Reviewer: Member 1 | 4 points | Integrates to: Gitea main

Depends on: S2-M3-1 and Member 5 work-log/timesheet endpoints

Goal: As a tutor, I want my existing work logs to become a submit/review timesheet workflow so that organisers can approve actual worked hours.

Acceptance checks

- Draft work logs can be grouped/submitted into the agreed timesheet state.
- Invalid hours, duplicate submissions and wrong-session ownership are rejected.
- Tutor sees pending/approved/returned status after organiser action.

Member tasks

- [] Build submit/review-status UI on top of Sprint 1 work-log creation.
- [] Connect real timesheet submit/status endpoints.
- [] Test draft, submit, duplicate, returned and approved cases.

Handoff: Member 2 consumes submitted timesheets; Member 6 validates totals/status constraints.

S2-M3-3 | Excuse status workflow and Tutor Basic regression

Branch: s2/m3/excuse-regression | Reviewer: Member 1 | 4 points | Integrates to: Gitea main

Depends on: S2-M3-1, S2-M3-2 and Member 5 excuse endpoint

Goal: As a tutor, I want the Sprint 1 excuse submission to show its later decision/status and remain reliable with the rest of the Basic journey.

Acceptance checks

- Submitted excuses show pending and later organiser outcome.
- Past/unowned sessions cannot be excused.
- Tutor regression covers dashboard, availability, work log, timesheet and excuse paths together.

Member tasks

- [] Add excuse status/history display rather than recreating submission.
- [] Connect organiser decision outcome once the endpoint exists.
- [] Run the full tutor Basic regression and fix reproducible defects.

Handoff: Coordinator receives a complete tutor demo script result; Member 2 sees excuse/availability impact.

Sprint 2 - Member 4

Student overflow plus shared responsive and accessible interface

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

CARRY FORWARD

Start from the latest Gitea main and reuse the working Sprint 1 implementation. The three cards below are the new work for this sprint.

S2-M4-1 | Connect Student overflow to real open-work API

Branch: s2/m4/overflow-api | Reviewer: Member 6 | 4 points | Integrates to: Gitea main

Depends on: Member 5 overflow endpoint and Member 6 schema

Goal: As a student, I want the Sprint 1 overflow shell to load real open work so that the mock boundary is removed without changing the established UI behavior.

Acceptance checks

- Only eligible/open work is returned for the signed-in Student.
- Closed/claimed work cannot be volunteered for.
- Loading, empty, error and retry states remain clear.

Member tasks

- [] Replace the Sprint 1 mock with the real student overflow endpoint.
- [] Keep the existing responsive list/cards and state components.
- [] Add integration tests for open, closed, empty and failure cases.

Handoff: Member 5 fixes response/permission issues; Member 6 supplies reproducible open/closed seed cases.

S2-M4-2 | Volunteer request and outcome statuses

Branch: s2/m4/volunteer-basic | Reviewer: Member 6 | 4 points | Integrates to: Gitea main

Depends on: S2-M4-1 and Member 5 volunteer endpoint

Goal: As a student, I want my confirmed volunteer request stored and its outcome shown so that I know whether I received the work.

Acceptance checks

- Confirm creates one request and duplicate clicks/claims are controlled.
- Pending/approved/rejected/already-claimed statuses are visible.
- Another Student cannot view or mutate the request.

Member tasks

- [] Connect the Sprint 1 confirmation UI to the real volunteer endpoint.
- [] Show persisted request/outcome states.
- [] Test duplicate, race, rejected and permission cases.

Handoff: Member 2 consumes pending claims; Member 5/6 enforce uniqueness and atomic outcome.

S2-M4-3 | External-result UI, accessibility and Student Basic regression

Branch: s2/m4/shared-quality | Reviewer: Member 6 | 4 points | Integrates to: Gitea main

Depends on: Member 6 external adapter and all Basic shared UI

Goal: As a user, I want the approved external holiday result/fallback and Student journey presented accessibly without weakening the working shared UI.

Acceptance checks

- External information is clearly supporting information and a safe fallback appears when unavailable.
- Student primary actions remain keyboard/mobile accessible.
- Student regression covers real overflow and volunteer flows.

Member tasks

- [] Render only the safe external API response/fallback from the backend.
- [] Run accessibility/responsive checks on Student and shared components.
- [] Fix Member 4-owned Basic regressions found in representative-user testing.

Handoff: Coordinator receives accessibility/test evidence; Member 6 receives any adapter failure cases.

Sprint 2 - Member 5

Handwritten Express API, validation, business rules, security and API tests

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

CARRY FORWARD

Start from the latest Gitea main and reuse the working Sprint 1 implementation. The three cards below are the new work for this sprint.

S2-M5-1 | Allocation-write and organiser-approval API completion

Branch: s2/m5/organiser-api | Reviewer: Member 2 | 4 points | Integrates to: Gitea main

Depends on: Member 6 Basic schema and Member 1 role matrix

Goal: As the application, we want real allocation mutations and organiser approval routes so Sprint 2 can complete workflows without redoing Sprint 1 course/tutor endpoints.

Acceptance checks

- Allocation create/edit/remove rechecks mark, clash and hours on the server inside safe database operations.
- Timesheet/claim organiser decisions enforce allowed state transitions.
- Legacy POST /api/tutors is not required by the normal self-service Tutor UI.

Member tasks

- [] Implement allocation mutation routes and transaction/service logic.
- [] Implement organiser timesheet/claim decision routes.
- [] Add validation, permission, conflict and state-transition tests.

Handoff: Member 2 connects the board; Member 6 tunes queries/constraints.

S2-M5-2 | Student overflow/volunteer and timesheet-status API

Branch: s2/m5/tutor-student-api | Reviewer: Member 2 | 4 points | Integrates to: Gitea main

Depends on: Member 6 Basic workflow schema and Member 1 ownership rules

Goal: As the frontend, we want the missing Student and workflow endpoints so the remaining Sprint 1 mocks/status gaps can become real.

Acceptance checks

- Student open-work and volunteer routes are role/ownership safe.
- Timesheet submit/status and excuse decision/status contracts are stable.
- Duplicate/race/state conflicts use safe 409-style responses.

Member tasks

- [] Implement Student overflow/volunteer endpoints.
- [] Complete timesheet/excuse status routes needed by Tutor/Organiser UI.
- [] Add success, validation, 401/403/404/409 tests.

Handoff: Members 3 and 4 connect their screens; Member 6 enforces database uniqueness/transitions.

S2-M5-3 | API contract, deployment and coverage hardening

Branch: s2/m5/api-quality | Reviewer: Member 2 | 4 points | Integrates to: Gitea main

Depends on: S2-M5-1, S2-M5-2 and CI from Member 6

Goal: As the team, we want the expanded Sprint 2 API to keep the existing safe error contract and remain deployable/testable as one service.

Acceptance checks

- New routes use the established { error: { code, message } } shape and do not leak stack traces/secrets.
- Hosted health/smoke behavior stays stable after deployment.
- Coverage includes critical success, permission, validation and conflict branches.

Member tasks

- [] Standardise only new/changed routes; do not rewrite already-stable Sprint 1 contracts unnecessarily.
- [] Run backend build/smoke/coverage and fix real regressions.
- [] Provide deployment and API evidence to the Integration Lead.

Handoff: Integration Lead uses health/tests/Codecov as merge evidence; Coordinator uses route examples for API docs.

Sprint 2 - Member 6

Neon PostgreSQL, Prisma migrations, external integration, data engine, CI/deployment

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

CARRY FORWARD

Start from the latest Gitea main and reuse the working Sprint 1 implementation. The three cards below are the new work for this sprint.

S2-M6-1 | Sprint 2 migrations for workflow state and allocation writes

Branch: s2/m6/basic-workflow-schema | Reviewer: Member 4 | 4 points | Integrates to: Gitea main

Depends on: Final Basic payloads from Members 2-5

Goal: As the team, we want only the new Sprint 2 data/state additions migrated so the working Sprint 1 schema is extended rather than recreated.

Acceptance checks

- New allocation mutation, timesheet/approval, excuse-status and volunteer-request data is represented with safe constraints.
- Migrations apply from the existing Sprint 1 schema and from a clean database.
- No manual production table edits are required.

Member tasks

- ☐ Add minimal versioned migrations for new Sprint 2 states/relations.
- ☐ Update seed data for approval and race/conflict cases.
- ☐ Run db verification and coordinate schema changes with API owners.

Handoff: Member 5 connects transactions/queries; Members 2-4 use stable demo data.

S2-M6-2 | Production public-holiday adapter with graceful fallback

Branch: s2/m6/external-api-adapter | Reviewer: Member 4 | 4 points | Integrates to: Gitea main

Depends on: Approved Sprint 1 external service and Member 5 backend

Goal: As Tutor Mx, we want the Sprint 1 holiday spike exposed safely through the backend with caching/fallback so the UI can use it without repeating the research spike.

Acceptance checks

- Only the backend calls the external service and returns the approved safe fields.
- Timeout/unavailable/invalid-response uses the established fallback.
- The integration does not block core Tutor Mx workflows.

Member tasks

- ☐ Turn the validated Sprint 1 spike into a production backend adapter/route.
- ☐ Add sensible caching and deterministic mocked tests.
- ☐ Document the fallback and any service limits.

Handoff: Member 5 exposes the adapter through handwritten API; Member 4 renders success/fallback.

S2-M6-3 | Allocation data performance, deployment smoke and Codecov reliability

Branch: s2/m6/data-delivery-quality | Reviewer: Member 4 | 4 points | Integrates to: Gitea main

Depends on: S2-M6-1, Member 5 API and Gitea Actions

Goal: As the Integration Lead, we want the completed Basic data flow and release checks reliable enough for repeated integration.

Acceptance checks

- Allocation candidates/writes and dashboard refresh use efficient queries with correct results.
- Clean setup/build/smoke works against the shared environment.
- Gitea Actions/coverage runs on the shared lecturer runners; queued jobs are expected and completed run evidence is retained.

Member tasks

- ☐ Profile/fix obvious allocation query issues and database constraints.
- ☐ Run clean migration/seed/check/smoke before release.
- ☐ Maintain Gitea CI/coverage config, verify the shared-runner result, and never invent green CI evidence.

Handoff: Integration Lead uses this as Sprint 2 merge gate; Coordinator receives run links and screenshots.

Sprint 2: integration and review gate

- [] Every Member starts from the latest Gitea main and hands over only their own tested branch.
- [] Assigned reviewer reproduces the issue-specific checks and checks page 7/8 evidence.
- [] Integration Lead integrates approved branches into main one at a time and reruns the affected regression/full gate.
- [] After a branch/main push, confirm Gitea Actions completes on a shared runner. A queued job is normal; wait for it. If infrastructure genuinely remains unavailable, record the blocker plus local evidence and do not mark CI green.
- [] After integration, run the relevant three-role/public-environment smoke test.
- [] Update the GitHub deployment mirror and manual Cloudflare Production artifact only from approved Gitea main when a release is required.
- [] Issue is Done only when integrated behavior passes; code merely existing on a Member branch is not Done.

SPRINT 2 SPECIFIC RELEASE CHECK

Verify real Student overflow/volunteer, allocation create/edit/remove and organiser approval/status journeys end to end.
Confirm no Sprint 1 manual-ID flow was reintroduced.

Post-Sprint 2 production stabilisation

15-16 September 2026 | Live feedback fixes that become the Sprint 3 baseline

HISTORICAL RECORD RULE

Sprint 1 and Sprint 2 issue cards remain the record of what was planned and delivered in those sprints. The items below were discovered through integrated/live testing after the Sprint 2 review window and are recorded as the baseline Sprint 3 inherits.

Why this work was needed

- The merged product worked end to end, but some cross-layer behaviour was confusing or inconsistent when tested as a real Tutor or Organiser.
- The team chose narrow fix branches with regression tests rather than reopening completed Sprint 1/2 foundations.
- The main themes were ownership (who sets a Tutor schedule/capacity), visibility (calendar/workload presentation), candidate API correctness and post-save state consistency.

Chronological changes

Date	Change	Result / new baseline
15 Sep	Final Sprint 2 stability pass	A failing organiser integration was reverted rather than shipped; passing cache/timesheet corrections were retained. Course deletion, organiser layout and timetable/candidate behaviour were retested.
15 Sep	Tutor-owned work preferences	Tutor sets weekly work capacity and Busy/Class/Study/Other unavailable periods. Organiser views this read-only.
15 Sep	Weekly calendar + detail resilience	Tutor and Organiser share a weekly schedule calendar. Marks and schedule load independently. Optional legacy time-slot fields no longer hide valid marks. Confusing UTC-week wording was removed.
16 Sep	Candidate API contract repair	The query already returned clashDetails, but server mapping omitted it. The mapping now includes clashDetails so the frontend validator accepts valid candidate responses.
16 Sep	Post-save self-conflict repair	The saved allocation ID is passed as excludeAllocationId during refresh so a newly created/edited confirmed allocation does not clash with itself. Genuine conflicts still block.
16 Sep	Empty-state clarification	Only tutors passing mark + availability + capacity are selectable. If none pass, the board states "No tutors currently available"; failed tutors remain read-only with reasons.
16 Sep	Direct week calendar navigation	Organiser workload keeps Previous / This week / Next controls and adds Go to week. Any chosen date resolves to its Monday-Sunday week and reloads the same server-supplied workload data.
16 Sep	Completed-session work-log gate	Tutor work logs are available only after the confirmed session occurrence has ended. Future/in-progress occurrences are disabled in the UI and the backend rejects early submissions; recurring sessions are checked per occurrence.

Sprint 3 carry-forward

- Preserve the completed Previous / This week / Next controls and Go to week date picker while adding ranking, shortages and reports.
- Build timetable import and recurring exceptions into the existing Tutor-owned calendar instead of creating a second schedule model.
- Preserve the completed-session work-log guard while extending timesheet corrections, disputes and payroll/export workflow.

Sprint 4 current-main code audit

30 September 2026 | Audited against the supplied current source archive **tutor-mx-system-main (5)(2).zip**. The archive has no .git directory, so this handbook does not claim a SHA.

WHY THE OLD ROADMAP IS REPLACED
The previous pages 30-50 were written before the latest shared timetable, fortnightly recurrence, Tutor sick-session logic and Master Organiser work. Sprint 4 now starts from this newer code and must not rebuild anything already present.

Area	What the current code already proves
Accounts/security	Auth0 Student/Tutor onboarding, approved Organiser sign-in, role guards, reset/delete, notifications/search, and Master Organiser approval routes/capability.
Organiser operations	Courses/tutors, staffing requirements, explainable ranking, bulk allocation, reports/budget/workload, approvals, overflow and Command Centre.
Tutor timetable	One manual/import calendar; CSV/ICS preview/commit; Class/Unavailable categories; Once/Weekly/Fortnightly recurrence; term data; calendar displays exception data.
Tutor attendance	Excuse request + Organiser decision; attendance state can become Excused or Skipped; sick work logs/timesheets are guarded.
Student today	Real overflow work, volunteer request/outcome/withdraw. Student does NOT yet have a personal timetable route, mutual availability booking or sick-note workflow.
Master Organiser	Lecturer applications stay pending without Organiser Profile/role; Master approval grants ORGANISER and creates/links the Profile.
Advanced Sprint 4	Scenarios/locks/presence, Tutor swaps, cross-system audit/restore and whole-school proposal intelligence are still the main Advanced gaps.

IMPORTANT DATABASE DETAIL
TimeSlot is already the schedule model, but its owner field is still tutor-named in the current schema. Sprint 4 must generalise that existing model for Students rather than creating StudentTimeSlot or a second recurrence/import engine.

Completed baseline before Sprint 4

Sprint 3 and the late September production fixes are now foundations. These are regression targets, not new Sprint 4 cards.

Completed area	Keep and reuse
Member 1 layer	Notification inbox/reminders, secure search and Ctrl/Cmd+K, guided Tutor setup.
Member 2 layer	StaffingRequirement, shortages, candidate ranking, bulk preview/commit, manual override reason, budget/workload/course-hour reporting and excuse review.
Member 3 layer	CSV/ICS/paste timetable import, AcademicTerm bounds, timesheet correction/revision/declaration/dispute and approved payroll export.
Member 4 layer	Student volunteer withdrawal plus Organiser overflow post/edit/close and requirement-to-overflow.
Member 5 layer	API permission/validation hardening, idempotency/write receipts, race/transaction protection and safe errors.
Member 6 layer	Command Centre KPIs, shortage heatmap, budget/workload/fairness visuals and Needs Attention feed.
Late timetable fixes	Manual/import tabs share the same schedule; fortnightly recurrence; category/series metadata; term publication/breaks; duplicate/import refresh fixes.
Late attendance fix	Tutor sick-session state is reflected in timetable/work-log eligibility while preserving history.
Late role fix	Master Organiser approval replaces ordinary self-created Organiser Profiles.

DO NOT REPEAT

Do not recreate current Tutor timetable/import/recurrence, Tutor sickness, Organiser registration approval, notifications/search, Command Centre, ranking, bulk allocation, reports or overflow. Sprint 4 extends these exact systems.

Student timetable - one shared calendar system

Main design rule: Student and Tutor use the same schedule engine. The UI changes by role, but recurrence/import/term/clash logic is shared.

DO THIS
Profile -> TimeSlot -> Recurrence/Term/Exceptions
A Student and a Tutor both own TimeSlot rows. Manual and imported entries go through the same validators and calendar expansion.

DO NOT DO THIS
Do not create StudentTimetable, StudentTimeSlot, StudentAcademicTerm, a second CSV parser, a second ICS parser or a second weekly-calendar component. Those would repeat working code and create inconsistent clash behaviour.

Data / service	Purpose after Sprint 4
TimeSlot (generalised owner)	Student/Tutor personal Class or Unavailable entries; Once/Weekly/Fortnightly; term bounds and occurrence exceptions.
Allocation (existing)	Organiser-assigned Tutor staffing work. Keep its meaning unchanged.
TutoringBooking (new)	Scheduling-only appointment between one Student and one Tutor. It appears on both calendars and blocks both from being double-booked. It does not replace Allocation/payroll.
Busy-interval service (shared)	Builds safe busy intervals from recurring TimeSlots plus role-specific sessions, then computes gaps. UI never duplicates this calculation.
Mutual availability	Student free intervals intersect Tutor free intervals. Return only safe bookable times, never the other person's private event labels.
StudentSickNote (new)	Student request linked to their own TutoringBooking; reuses Organiser approval/status patterns from the Tutor excuse flow without rewriting the Tutor table.

PRIVACY RULE
A Student may learn that a Tutor is Busy/Free for booking, but must not receive the Tutor's private class/event names. A Tutor must not see a Student's private timetable labels just because they share availability.

New Student experience - what must work by Sprint 4 end

Tutor timetable	Student timetable
University/classes	University/classes
Organiser allocations + tutoring bookings	Tutoring sessions the Student booked
Tutor capacity/work preferences	No Tutor capacity setting; only personal availability
Manual events	Manual events
CSV/ICS import	CSV/ICS import
Once/Weekly/Fortnightly + term/exception rules	Same recurrence/term/exception rules
Used for Tutor clash/capacity checks	Used to find when the Student is free to book

SHARED AVAILABILITY EXAMPLE
Student free AND Tutor free = bookable.
If the Student is free 11:00-14:00 but the Tutor is busy 12:00-13:00, the safe booking choices are 11:00-12:00 and 13:00-14:00 (subject to requested session length). The backend performs this intersection and rechecks it again when the booking is saved.

STUDENT SICK NOTE - SAME APPROVAL METHOD AS TUTOR
Student opens one of their own booked tutoring sessions -> submits a sickness reason -> status is Pending -> Organiser approves/rejects in the same approvals workspace pattern -> Student sees the outcome. Approved in advance can release the future booking as Excused; an approved request on/after the session stays in history as Skipped. Do not silently delete history.

NO EXTRA MEDICAL-UPLOAD SYSTEM
The current Tutor sickness flow stores a reason/status, not a document upload. "Same as Tutor" therefore means the same kind of request/decision/status workflow. Do not invent file storage unless the course brief separately requires it.

Sprint 4: updated plan, order and handovers

30 September-11 October 2026 | Advanced features plus the new Student timetable/booking/sickness scope, all built on the current working system.

EXACT CHRONOLOGICAL METHOD
Member 1 starts from current approved main. They finish one branch and tell the Integration Lead. The Integration Lead reviews/tests/merges it. **Only then Member 2 pulls updated main and starts.** Repeat M1 -> merge -> M2 -> merge -> M3 -> merge -> M4 -> merge -> M5 -> merge -> M6. This merge step is workflow, not a separate Member task.

Member	Only new Sprint 4 work
Member 1	Generalise the existing timetable owner for Student/Tutor; Student My Timetable + manual/import entry; production session/stale-target hardening; notification actions + recent/favourites.
Member 2	Draft scenarios/locks/Compare with Live/Publish; optimistic concurrency + Organiser presence + scenario library; mutual Student/Tutor availability service and read-only booking-time panel.
Member 3	Tutor swap workflow; Student tutoring booking creation/cancel/history using Member 2 mutual times; both calendars update and booking becomes part of clash checks.
Member 4	Student sick-note request + Organiser approval/status using Tutor-like method; race/conflict/notification polish; final responsive/accessibility/table alternatives.
Member 5	Cross-system audit timeline; allocation restore; filters/detail/export; audit/security/idempotency hardening for new booking and Student sick-note mutations.
Member 6	Whole-school explainable proposal engine; Why this Tutor?/Why not? cockpit; strategy comparison/impact lab. Reuse shared busy/clash rules including confirmed Student bookings.

NO-FORWARD-DEPENDENCY RULE
Each Member may depend on work already merged before their turn. A Member must not wait for a later Member to add a route/table/service they need. If a small supporting change is required for that Member's feature, it belongs on the same branch.

Sprint 4 - Member 1

BEGINNER-FRIENDLY SUMMARY: First, make the existing timetable truly shared so Students can use the same manual/import/calendar engine as Tutors. Then harden final login/session and stale search/notification behaviour. Do not rebuild Auth0, Tutor timetable import, notifications or search.

STARTING POINT / DO NOT REPEAT

Start from approved current main. Reuse TimeSlot, AcademicTerm, timetable import, recurrence, calendar, notifications and search. The current TimeSlot owner name is Tutor-specific; generalise it instead of adding a second Student schedule model.

S4-M1-1 | Shared TimeSlot ownership + Student My Timetable

BEGINNER GOAL	Generalise the working schedule model so a Student can add/import their own classes and unavailable time using the exact same recurrence, term and calendar rules as a Tutor.
USER STORY	As a Student , I want my classes and unavailable time in Tutor Mx so that the system knows when I am free for tutoring.
ACCEPTANCE TESTS	- One TimeSlot system serves Student and Tutor; existing Tutor rows migrate intact. - Student can add manually and import CSV/ICS with the existing preview/duplicate/term rules. - Student supports Once, Weekly and Fortnightly and sees the same weekly calendar expansion. - A Student can read/change only their own entries; another user/role cannot access them. - Existing Tutor timetable and allocation clash behaviour still passes.
TASKS	- [] Add a versioned migration that changes Tutor-specific TimeSlot ownership naming to a generic Profile owner (or equivalent minimal generic relation). - [] Add shared own-schedule routes/services; keep Tutor route aliases if needed for compatibility. - [] Add /student/timetable navigation and reuse MyTimetable/AvailabilityForm/TimetableImport rather than cloning them. - [] Add Student/Tutor ownership, import, recurrence and regression tests.

S4-M1-2 | Production account/session + stale-target hardening

BEGINNER GOAL	Make expired/revoked sessions and deleted/forbidden search or notification links fail safely in the finished product.
USER STORY	As any signed-in user , I want expired access or stale links handled clearly so that old browser state cannot expose or change protected data.
ACCEPTANCE TESTS	- 401/403 handling clears only intended protected state and never leaves a false signed-in state. - Opening an old search/notification target rechecks permission and shows safe unavailable/not found state. - Fresh search no longer returns newly forbidden/deleted records.
TASKS	- [] Audit session/storage cleanup and production-like reset/delete/sign-out failures. - [] Reproduce stale/deleted/changed-role targets and fix only real gaps. - [] Add focused auth, permission and cache-refresh regression.

S4-M1-3 | Actionable notifications + Ctrl/Cmd+K recent/favourites

BEGINNER GOAL	Add safe action links and recent/favourite destinations to the existing inbox/palette, including the new Student timetable destination.
USER STORY	As a Tutor Mx user , I want to jump directly back to permitted work so that common tasks are quick without weakening permissions.
ACCEPTANCE TESTS	- Notification actions and palette destinations always recheck current permission/state. - Recent/favourite entries are role-safe, de-duplicated and keyboard usable. - Student timetable can be a safe recent/favourite destination; private record data is not cached into the shortcut.
TASKS	- [] Add safe View/Fix/Review links to existing notification cards. - [] Add recent/favourite destination metadata to the existing palette. - [] Test keyboard focus, stale target, wrong-role and removed destination cases.

ONE-BRANCH HANDOFF

Keep all Member 1 Sprint 4 work on **s4/m1/completion**. Self-test only this branch, push it and hand evidence to the Integration Lead. Do not merge to main. The Integration Lead reviews/tests and merges it to main. Only then does Member 2 pull updated main and create s4/m2/completion.

Sprint 4 - Member 2

BEGINNER-FRIENDLY SUMMARY: Build the Organiser planning sandbox, then add the read-only Student/Tutor shared-availability calculation. Students should be able to see safe mutual booking times, but this branch does not save a booking yet.

STARTING POINT / DO NOT REPEAT

Start from main after Member 1 is merged. Reuse StaffingRequirement, ranking, hard rules, reports, Command Centre and the newly shared TimeSlot owner. Scenario/version/lock/presence storage and mutual-availability service belong on this Member 2 branch.

S4-M2-1 | Draft scenarios, locks, Compare with Live and Publish

BEGINNER GOAL	Let Organisers experiment with alternative allocation plans without changing live Allocation rows until explicit Publish.
USER STORY	As an Organiser , I want named draft allocation scenarios so that I can compare choices safely before changing live staffing.
ACCEPTANCE TESTS	- Draft changes remain separate from live allocations. - Compare shows added/removed/changed assignments plus shortage/workload/budget differences. - Locks keep organiser-chosen assignments unchanged. - Publish rechecks hard rules and is all-or-nothing.
TASKS	- [] Add Scenario/ScenarioItem/lock storage and API/UI. - [] Build side-by-side diff and explicit Publish confirmation. - [] Test locked items, stale scenario, publish failure, rename/delete.

S4-M2-2 | Optimistic concurrency, visible Organiser presence + scenario library

BEGINNER GOAL	Prevent silent overwrite when two Organisers edit the same planning context and make draft scenarios easy to organise.
USER STORY	As an Organiser , I want to see active collaborators and receive a clear conflict on stale saves so that another person's newer work is not overwritten.
ACCEPTANCE TESTS	- Version/updatedAt is required on versioned writes; stale write returns conflict. - Presence is a short-lived activity indicator, not permission authority. - Scenario library supports rename, duplicate, notes, archive/reopen without touching live allocations.
TASKS	- [] Add optimistic version checks and presence heartbeat/TTL. - [] Show last updated/by plus refresh/compare/retry path. - [] Add scenario library actions and two-browser tests.

S4-M2-3 | Mutual Student/Tutor availability service + booking-time panel

BEGINNER GOAL	Calculate safe times where both people are free using the shared schedule engine, without revealing either person's private timetable labels.
USER STORY	As a Student , I want to select a Tutor and see only times when we are both free so that I can choose a realistic tutoring slot.
ACCEPTANCE TESTS	- Student busy time uses own recurring TimeSlots and existing Student bookings; Tutor busy time uses TimeSlots, confirmed allocations and bookings. - Term bounds, breaks, recurrence exceptions and requested session length are respected. - API returns free/bookable intervals, not private event names. - Search window/result count is bounded and only future slots are offered. - This card reads availability only; it does not create the booking yet.
TASKS	- [] Add a shared busy-interval/gap service and role-safe mutual-availability endpoint. - [] Add Student-safe Tutor/course discovery needed by this screen on the same branch if current search is insufficient. - [] Add a Student shared-availability table/panel using backend results. - [] Test privacy, recurrence, overlaps, no-common-time and boundary duration cases.

ONE-BRANCH HANDOFF

Keep all Member 2 Sprint 4 work on **s4/m2/completion**. Self-test only this branch, push it and hand evidence to the Integration Lead. Do not merge to main. The Integration Lead reviews/tests and merges it to main. Only then does Member 3 pull updated main and create **s4/m3/completion**.

Sprint 4 - Member 3

BEGINNER-FRIENDLY SUMMARY: Build two real scheduling workflows: Tutor swaps and Student tutoring bookings. Member 2 already shows mutual free times; now make one of those times safely become a booking that appears on both calendars.

STARTING POINT / DO NOT REPEAT

Start from main after Member 2 is merged. Reuse the same mark/clash/capacity rules for swaps and the mutual-availability service for Student bookings. Do not create another timetable engine or recompute free time in React.

S4-M3-1 | Tutor swap request -> replacement decision -> Organiser decision

BEGINNER GOAL	Let a Tutor propose a future-session replacement, let the replacement Tutor accept/decline, and let the Organiser make the final decision after a hard-rule recheck.
USER STORY	As a Tutor, I want to request a safe replacement for a future session so that unavoidable conflicts can be handled without silently changing allocations.
ACCEPTANCE TESTS	- Replacement eligibility reuses normal mark/clash/selected-week capacity rules. - Live allocation does not change when the request is created. - Replacement Tutor sees only requests addressed to them. - Organiser approval rechecks rules/version and applies the change atomically. - History shows request/accept/decline/approve/reject/cancel.
TASKS	- [] Add Swap storage, proposal/cancel UI/API and notifications. - [] Add replacement accept/decline and Organiser approve/reject. - [] Add before/after calendar/hour preview and immutable history. - [] Test duplicate, stale, changed capacity/clash and concurrent allocation change.

S4-M3-2 | Student tutoring booking from a mutual free slot

BEGINNER GOAL	Turn one Member 2 mutual free interval into a real scheduling-only appointment between the signed-in Student and selected Tutor.
USER STORY	As a Student, I want to book one of our mutually free times so that the tutoring session appears in both calendars and neither person can be double-booked.
ACCEPTANCE TESTS	- New TutoringBooking stores Student, Tutor, course, start/end and status; it does not replace organiser Allocation/payroll. - Server rechecks mutual availability inside the write transaction before confirming. - Confirmed booking appears on both Student and Tutor timetables and becomes busy time for later clash checks. - Another Student cannot book the same Tutor/time after it becomes unavailable. - Student/Tutor cannot view another person's private booking beyond what their role needs.
TASKS	- [] Add minimal TutoringBooking migration/model and protected read/write service. - [] Add Book this time confirmation from the mutual-availability panel. - [] Merge confirmed bookings into both calendar projections and shared busy-interval service. - [] Add transaction/race, ownership, double-booking and calendar tests.

S4-M3-3 | Booking detail, cancellation, history and notifications

BEGINNER GOAL	Make the new Student booking understandable after it is created and allow safe future cancellation without deleting history.
USER STORY	As a Student or Tutor, I want clear booking details and status changes so that I know what is scheduled and what changed.
ACCEPTANCE TESTS	- Both sides see course, date/time and current status; no private timetable labels are exposed. - Allowed future cancellation changes status rather than deleting the record and immediately frees the interval. - Stale cancellation returns conflict and refreshes to server truth. - Affected Student/Tutor receive one de-duplicated notification.
TASKS	- [] Add booking list/detail/history to the existing timetable experience. - [] Add defined cancellation rule and version check. - [] Emit/reuse notifications for confirm/cancel. - [] Test cancel timing, stale state, repeat click and calendar refresh.

ONE-BRANCH HANDOFF

Keep all Member 3 Sprint 4 work on **s4/m3/completion**. Self-test only this branch, push it and hand evidence to the Integration Lead. Do not merge to main. The Integration Lead reviews/tests and merges it to main. Only then does Member 4 pull updated main and create s4/m4/completion.

Sprint 4 - Member 4

BEGINNER-FRIENDLY SUMMARY: Add the Student sick-note workflow using the same kind of Organiser approval the Tutor already has, then polish races, accessibility and dense planning screens. The Student feature must be complete on this branch - do not wait for Member 5 to build its API.

STARTING POINT / DO NOT REPEAT
Start from main after Member 3 is merged. Reuse the Tutor excuse status/approval interaction pattern, Organiser Approvals screen, shared notifications and the new TutoringBooking. Keep the current Tutor Excuse table/logic working; do not rewrite it just to add Students.

S4-M4-1 | Student sick note + Organiser approval/status

BEGINNER GOAL	Let a Student report sickness against their own tutoring booking and receive the same clear Pending/Approved/Rejected decision pattern used by Tutors.
USER STORY	As a Student , I want to mark a booked tutoring session as sick and see the Organiser decision so that my absence is recorded instead of silently deleting the session.
ACCEPTANCE TESTS	- Student can submit only for their own eligible TutoringBooking; one active request per booking. - Request stores reason and starts PENDING; Organiser Approvals shows a Student sick-note queue using the same approve/reject/review-note pattern. - Approved in advance marks the future appointment Excused and releases it from active busy time; approved on/after the session is shown as Skipped. History remains visible. - Rejected request leaves the booking scheduled and shows the review note/status. - No medical file upload is introduced because the existing Tutor flow does not have one.
TASKS	- [] Add StudentSickNote (or minimal equivalent) linked to TutoringBooking with PENDING/APPROVED/REJECTED and review fields. - [] Extract/reuse shared absence decision/UI helpers where practical instead of copy-paste Tutor logic. - [] Add Student submit/status UI and Organiser approval queue/actions. - [] Add notifications, ownership/state/duplicate tests and timetable busy-state tests.

S4-M4-2 | Two-session race/conflict + notification polish

BEGINNER GOAL	Make competing overflow, booking, cancellation and sick-note actions converge on one server truth with clear refresh/retry messages.
USER STORY	As a Student/Organiser , I want stale actions explained so that the screen never shows an impossible state after another user acted first.
ACCEPTANCE TESTS	- Competing actions result in one legal persisted state. - Stale UI refreshes to server truth and hides actions that are no longer valid. - The same event/outcome does not create duplicate notifications.
TASKS	- [] Run two-browser race cases for overflow plus new booking/sick-note mutations. - [] Polish shared conflict banner/dialog and focus recovery. - [] Fix duplicate/irrelevant notification display only where reproduced.

S4-M4-3 | Final responsive/accessibility polish + table/list alternatives

BEGINNER GOAL	Keep Student/Tutor/Organiser screens usable on phone/tablet/desktop and by keyboard, including dense scenario/proposal/audit visuals.
USER STORY	As any user , I want the same information and actions available without clipped controls or colour-only meaning so that Tutor Mx is understandable on different devices and access methods.
ACCEPTANCE TESTS	- No whole-page horizontal scrolling or clipped primary actions in changed screens. - Dialogs restore focus; status/conflict changes are announced with text. - Key heatmap/chart/proposal data has an equivalent labelled table/list view. - Compact view preserves the same underlying values and actions.
TASKS	- [] Audit new Student timetable/booking/sick screens plus dense Organiser screens. - [] Add table/list alternative and compact mode to key visuals. - [] Run keyboard/focus/labels/contrast/long-text/common-width tests and capture evidence.

ONE-BRANCH HANDOFF
Keep all Member 4 Sprint 4 work on **s4/m4/completion**. Self-test only this branch, push it and hand evidence to the Integration Lead. Do not merge to main. The Integration Lead reviews/tests and merges it to main. Only then does Member 5 pull updated main and create s4/m5/completion.

Sprint 4 - Member 5

BEGINNER-FRIENDLY SUMMARY: Make important changes traceable and recoverable. Build one audit timeline and safe allocation restore, then harden the new Student booking/sick-note mutation APIs for permissions, retries and races. You are not the person who merges/releases main.

STARTING POINT / DO NOT REPEAT
Start from main after Member 4 is merged. Scenario, presence, swap, Student timetable, booking and Student sick-note routes already exist. Member 5 instruments/hardens them; Member 5 does not rebuild their feature UI.

S4-M5-1 | Cross-system audit trail + Command Centre timeline

BEGINNER GOAL	Create one safe audit service that records important successful changes with actor, time, action, target and redacted before/after detail.
USER STORY	As an Organiser , I want to see who changed important Tutor Mx records and why so that decisions can be investigated without reading raw database rows.
ACCEPTANCE TESTS	- Audit is written only after a successful committed mutation. - Scenario/publish, allocation, swap, booking status, Student sick-note decision and restore actions use the shared audit pattern where relevant. - Tokens/secrets/private raw request bodies are never stored. - Reasons from manual choice/return/reject/restore remain visible when appropriate.
TASKS	- [] Add AuditEvent/minimal snapshot schema and shared redaction/builder service. - [] Instrument important mutation routes after successful transaction. - [] Add Organiser timeline with context filters and permission tests. - [] Test rollback/no-audit and actor attribution.

S4-M5-2 | Allocation restore preview -> restore as a new change

BEGINNER GOAL	Let an Organiser compare current allocation state with a historical audited version and safely restore only if current hard rules still pass.
USER STORY	As an Organiser , I want to restore a valid earlier allocation without deleting history so that mistakes can be corrected transparently.
ACCEPTANCE TESTS	- Preview shows current vs historical values before confirmation. - Restore rechecks version + mark/clash/capacity and refuses stale/invalid target state. - Successful restore creates a new mutation and audit event; old history remains immutable.
TASKS	- [] Add history/preview/restore endpoint and UI. - [] Use transaction + expected version. - [] Test success, hard-rule fail, stale version, permission and rollback.

S4-M5-3 | Audit filters/export + new Student API hardening

BEGINNER GOAL	Make audit history usable and make the new timetable/booking/sick-note writes safe under invalid input, retries and competing requests.
USER STORY	As the team , we want bounded, role-safe history and mutation contracts so that final Sprint 4 features fail predictably instead of duplicating or leaking data.
ACCEPTANCE TESTS	- Audit filters by date/actor/action/target and safe CSV export preserves redaction. - Student schedule/booking/sick-note routes have bounded inputs, consistent safe errors and cross-user 403/non-revealing 404 behaviour. - Retry-prone booking/sick decisions use version/idempotency strategy where needed. - Focused concurrency tests prove no duplicate booking or double decision is persisted.
TASKS	- [] Add audit filter/detail/export tools. - [] Extend endpoint-role/ownership matrix to all new Student/Scenario/Swap/Audit routes. - [] Add missing 400/401/403/404/409/idempotency/concurrency tests. - [] Record focused API evidence for handover; do not run the final release gate.

ONE-BRANCH HANDOFF
Keep all Member 5 Sprint 4 work on **s4/m5/completion**. Self-test only this branch, push it and hand evidence to the Integration Lead. Do not merge to main. The Integration Lead reviews/tests and merges it to main. Only then does Member 6 pull updated main and create s4/m6/completion.

Sprint 4 - Member 6

BEGINNER-FRIENDLY SUMMARY: Build the final planning intelligence. Generate explainable whole-school draft allocations, explain Why this Tutor?/Why not?, and compare strategy presets. Reuse every existing hard rule, lock, scenario, audit and busy-time source - including confirmed Student tutoring bookings.

STARTING POINT / DO NOT REPEAT

Start from main after Member 5 is merged. Reuse StaffingRequirement, ranking, scenarios, locks, budgets, audit, live Allocation and the shared busy/clash provider. Member 6 creates proposal drafts only; it does not deploy or write directly to live allocations.

S4-M6-1 | Whole-school explainable proposal engine + objective presets

BEGINNER GOAL	Generate a deterministic draft staffing plan across open requirements while never breaking hard rules or Organiser locks.
USER STORY	As an Organiser , I want Tutor Mx to propose a complete draft plan so that I can review a strong starting point instead of allocating every requirement manually.
ACCEPTANCE TESTS	- Hard constraints remain mark + clash + selected-week capacity; locks never move. - Confirmed Student tutoring bookings count as Tutor busy time through the same clash service. - Soft presets may weight shortage first, stronger marks, fair workload and budget/spend; presets change weights only. - Proposal output is a normal Scenario, never live Allocation. - Same inputs + preset produce deterministic output.
TASKS	- [] Build proposal engine from existing eligibility/ranking/requirement/report data. - [] Add Balanced, Strongest Match, Fair Workload and Budget Aware preset weights. - [] Test no solution, tie, lock-heavy, student-booking clash, budget pressure and week boundary.

S4-M6-2 | Proposal cockpit: Why this Tutor? / Why not?

BEGINNER GOAL	Make every suggested or skipped Tutor understandable in plain language and connect the proposal to the existing scenario workflow.
USER STORY	As an Organiser , I want to understand why each Tutor was chosen or skipped so that I can review the proposal rather than trust a black box.
ACCEPTANCE TESTS	- Final path is Generate Proposal -> Why these Tutors? -> Save as Scenario -> Compare with Live -> Publish. - Why this Tutor shows hard-rule pass + soft factors; Why not distinguishes hard failure from lower soft score. - Shortage/budget/workload numbers use the same Sprint 3 report definitions. - Manual change is allowed only to another eligible Tutor and requires a reason.
TASKS	- [] Build proposal cockpit, explanation drawer and scenario handoff. - [] Reuse ranking/diff/audit/visual components instead of cloning them. - [] Test explanation accuracy, stale data, keyboard and manual eligible adjustment.

S4-M6-3 | Strategy comparison + proposal impact lab

BEGINNER GOAL	Run the same school data through several soft-objective presets and compare coverage, shortage, spend and workload side by side.
USER STORY	As an Organiser , I want to compare different valid planning strategies so that I can choose the draft that best fits the current priority.
ACCEPTANCE TESTS	- Presets share identical input and hard rules/locks. - Side-by-side metrics reconcile with existing shortage/budget/workload definitions. - Choosing a result saves/opens a draft Scenario and never directly publishes live. - Same input + same preset remains deterministic.
TASKS	- [] Add multi-preset comparison UI and impact cards/table. - [] Add explanation links and Save chosen proposal as Scenario. - [] Test ties, no-solution, lock-heavy and budget-pressure cases.

ONE-BRANCH HANDOFF

Keep all Member 6 Sprint 4 work on **s4/m6/completion**. Self-test only this branch, push it and hand evidence to the Integration Lead. Do not merge to main. The Integration Lead reviews/tests and merges it to main. After this handover, the Integration Lead completes the final combined-main checks and release workflow.

Sprint 4: chronological merge and final release gate

This is a workflow checklist for the Integration Lead, not a seventh task and not a Member 1-6 card.

EXACT SEQUENCE

Member 1 hands over -> Integration Lead reviews/tests/merges -> Member 2 pulls updated main -> repeat through Member 6. Members never merge themselves. The next Member never starts from the previous Member feature branch.

After each Member handover

- Review only that Member branch against its user story/acceptance tests.
- Run the affected regression on updated main after merge.
- Resolve merge/integration conflicts with the author; do not silently drop another Member's work.
- Confirm main is healthy before telling the next Member to pull and start.

Final behaviour that must be demonstrable after Member 6

- Master Organiser: lecturer registration -> pending -> approve/reject -> approved Organiser sign-in; normal Organiser cannot access the Master queue.
- Student timetable: manual/import classes, Once/Weekly/Fortnightly, terms/exceptions, same calendar engine as Tutor.
- Mutual availability: safe Student free INTERSECT Tutor free -> only valid booking times.
- Tutoring booking: server recheck, both calendars update, double-booking blocked, cancellation/history/notifications work.
- Student sick note: own booking -> Pending -> Organiser approve/reject -> Student sees outcome; approved advance release vs skipped history works.
- Existing Tutor sickness, overflow, ranking/bulk, reports, timesheets, notifications/search and Command Centre still work.
- Advanced Sprint 4: scenarios/locks/presence/conflicts, Tutor swaps, audit/restore and explainable proposal/strategy comparison work end-to-end.

FINAL RELEASE PROOF

Run the full automated gate + coverage + clean-browser Student/Tutor/Organiser/Master-Organiser smoke, database migration/recovery checks, security/idempotency/concurrency tests, then sync approved Gitea main to the deployment mirror and deploy/verify Render + Cloudflare from that approved source. Record the release SHA/evidence in the real repository.

Final requirement traceability - updated

Requirement / goal	Current baseline	Sprint 4 owner
Student personal timetable using existing infrastructure	Tutor TimeSlot/import/calendar exists; Student route missing	M1 generalises TimeSlot owner + Student UI
Student/Tutor shared availability	Tutor clashes exist; Student schedule intersection missing	M2 shared busy-interval + mutual availability
Student tutoring session booking	No Student booking model today	M3 TutoringBooking + both calendars
Student sick note with Organiser approval	Tutor Excuse workflow exists; Student equivalent missing	M4 StudentSickNote + shared approval pattern
Production session/stale link hardening	Auth/search/notification systems exist	M1
Draft scenarios + locks + publish + live presence	Missing	M2
Tutor swaps	Missing	M3
Race/accessibility/table alternatives	Partial shared UI exists	M4
Audit trail + allocation restore + export	Missing	M5
Whole-school proposal + explanations + strategies	Ranking exists, whole-school proposal missing	M6
Master Organiser approval	Already in current code	Regression only - no Sprint 4 rebuild
Tutor timetable/fortnightly/import/sick flow	Already in current code	Regression only - reused by new Student work

COVERAGE CHECK
By the end of Member 6, the new Student timetable/booking/sickness request is complete in addition to the original Sprint 4 Advanced scope. No Member card is a future placeholder: every earlier card includes any small route/schema/UI needed to finish its own feature.

Suggested final demo flow - now includes Student scheduling

- **Master Organiser:** register a lecturer -> show Pending -> Master Organiser opens Organiser applications -> approve -> lecturer signs in as Organiser.
- **Student timetable:** sign in as Student -> add one class manually -> import a weekly/fortnightly timetable row -> show both on My Timetable.
- **Mutual availability:** choose a Tutor -> show a simple Free/Busy table -> point out that only shared free times are bookable and private event names are hidden.
- **Booking:** book a mutual free time -> switch to Tutor -> show the same appointment on the Tutor timetable -> attempt a conflicting second booking and show it is blocked.
- **Student sick note:** Student submits sickness for the booking -> Organiser opens Approvals -> approve/reject -> Student timetable/status updates while history remains.
- **Command Centre:** show live staffing, shortage, budget/workload and Needs Attention, then create two draft scenarios and demonstrate locks/presence/conflict handling.
- **Tutor swap:** request replacement -> replacement accepts -> Organiser approves -> calendars/history update.
- **Proposal:** Generate Allocation Proposal -> Why this Tutor?/Why not? -> compare strategy presets -> Save as Scenario -> Compare with Live -> Publish.
- **Audit:** open timeline -> filter a change -> preview allocation restore -> restore as a new auditable event.
- **Release proof:** show approved main/release SHA and live /health + /ready plus Student/Tutor/Organiser smoke.

DEMO STORY TO EXPLAIN

Tutor Mx now uses the same scheduling rules for both people. A Student timetable answers "When am I free to have tutoring?"; a Tutor timetable answers "When am I free to tutor?". The system intersects those two answers, then rechecks the chosen slot before saving a booking.

SCOPE CONTROL

Do not add unrelated AI chat, gamification or decorative dashboards. The strongest final features are the ones that connect existing roles, schedules, approvals and allocation rules into one defensible workflow.

Final beginner memory aid

The short version for every team member

- One Member = one Sprint 4 branch: s4/m1/completion through s4/m6/completion.
- Always start from updated main. Member 2 starts only after Member 1 is merged; continue in exact order through Member 6.
- The Integration Lead merge between Members is a workflow step, not a Member card and not extra Sprint scope.
- No forward dependencies: if your feature needs a small migration/route/service/UI change, include it on your own branch.
- Do not rebuild working features. Tutor timetable/import/fortnightly/terms, Tutor sickness, Master Organiser, Command Centre, search/notifications, staffing/ranking/bulk/reports/overflow are current foundations.
- Student timetable must reuse TimeSlot/import/calendar. No StudentTimeSlot or second recurrence/parser system.
- Shared availability is calculated on the backend from both people's busy intervals and exposes Free/Busy, not private event labels.
- TutoringBooking is a scheduling record that appears on both calendars and blocks double-booking; it does not replace organiser Allocation/payroll.
- Student sick note copies the Tutor approval method, not the Tutor database table: own session + reason + Pending/Approved/Rejected + Organiser decision + preserved history.
- Hard allocation rules remain fixed: mark AND clash AND selected-week capacity. Confirmed Student bookings are busy time for the Tutor.
- Advanced completion still means scenarios/locks/presence/conflicts, swaps, audit/restore and explainable proposal/strategy comparison.
- After your branch: focused local tests/browser acceptance -> push -> wait for branch CI/Codecov -> hand evidence to Integration Lead. Do not merge main yourself.

FINAL RULE

Before implementing any card, inspect current main. If another merged change already solved part of your card, keep only the remaining gap. The goal is one coherent Tutor Mx system, not six parallel implementations.

SOURCE NOTE

This edition uses the supplied 18 September handbook plus the supplied 30 September source archive. Pages 13-29 are retained historical evidence. Pages 30 onward replace the old future Sprint 3-4 roadmap with the current completed baseline and updated Sprint 4 plan.